

Day 3 Assignment

Building a To-Do List App with Copilot, ChatGPT and CodeWhisperer

John Michael Bilbao
Techstart
August 11, 2026

Introduction

The assignment asks for a small Python to-do list app built with three AI tools, and a description of what each one actually contributed. I used Copilot for the interface, ChatGPT for the function that adds a task, and CodeWhisperer to review what the other two produced.

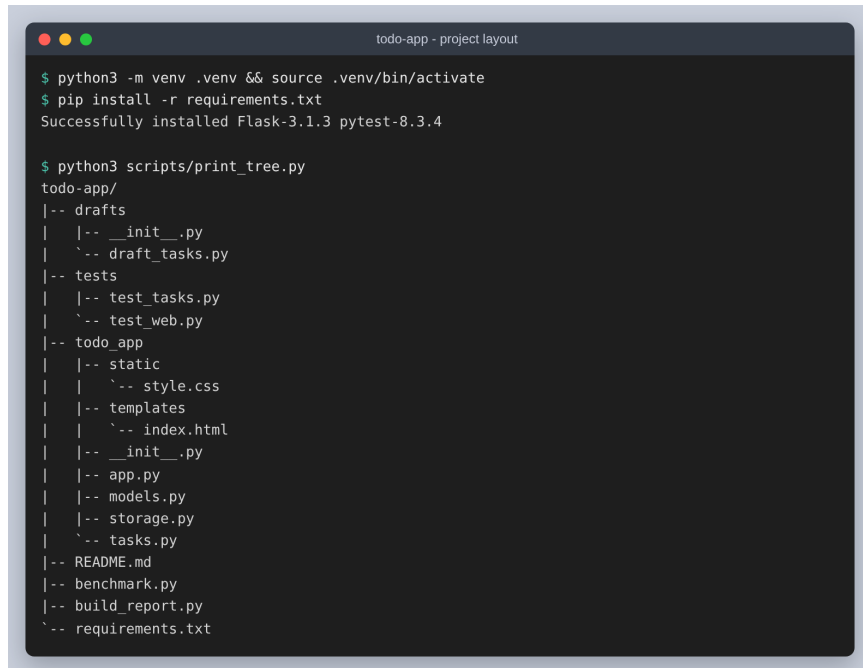
I built it as a Flask web app rather than a console script. The interface is then ordinary HTML, which is the kind of repetitive markup Copilot is good at, and the finished app is something I can point a browser at and photograph. The whole thing is about 300 lines across five Python files, one template and one stylesheet.

The short version of what happened: Copilot was the fastest of the three but needed watching, ChatGPT wrote the best single function, and CodeWhisperer found real problems that neither of the other two noticed. It also gave me one piece of advice that made the code slower when I followed it literally, which took a benchmark to notice.

Step 1: Setting up the environment

I worked in VS Code with the GitHub Copilot extension and the AWS Toolkit, which is what CodeWhisperer ships inside. ChatGPT I used in the browser rather than through an extension, because I wanted to be able to argue with it about the design before taking any code.

The project is a virtual environment with Flask and pytest in it, and the layout below. Splitting storage from the task operations was a decision I made before asking any of the tools for anything, and it turned out to matter later: because the file handling sits in one class, the fixes that came out of the review touched one file instead of being spread through the app.



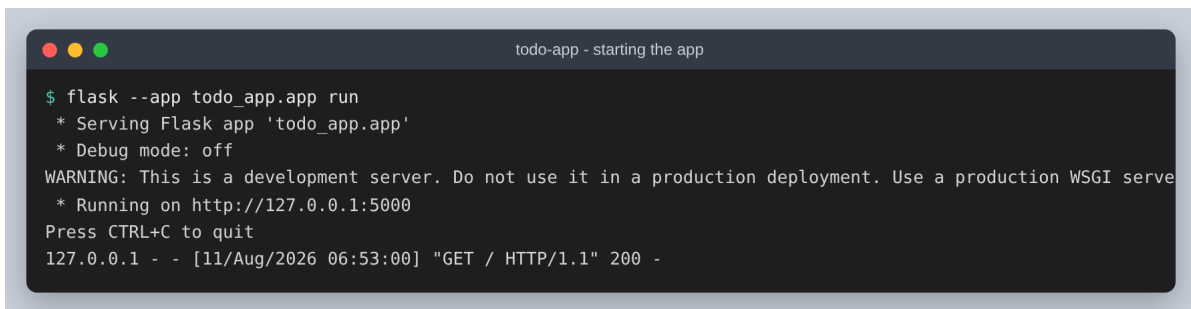
```
todo-app - project layout

$ python3 -m venv .venv && source .venv/bin/activate
$ pip install -r requirements.txt
Successfully installed Flask-3.1.3 pytest-8.3.4

$ python3 scripts/print_tree.py
todo-app/
|-- drafts
|   |-- __init__.py
|   |-- draft_tasks.py
|-- tests
|   |-- test_tasks.py
|   |-- test_web.py
|-- todo_app
|   |-- static
|   |   |-- style.css
|   |-- templates
|   |   |-- index.html
|   |-- __init__.py
|   |-- app.py
|   |-- models.py
|   |-- storage.py
|   |-- tasks.py
|-- README.md
|-- benchmark.py
|-- build_report.py
-- requirements.txt
```

Figure 1: The application files. The scripts that build this write-up and its figures live alongside them and are left out here.

Flask serves the app on port 5000, and the whole interface is a single page that is re-rendered after every action.



```
todo-app - starting the app

$ flask --app todo_app.app run
* Serving Flask app 'todo_app.app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [11/Aug/2026 06:53:00] "GET / HTTP/1.1" 200 -
```

Figure 2: Starting the app

Step 2: Using Copilot for the interface

I wrote the stylesheet first, with the class names I wanted, then opened the template and started typing. That order matters. Copilot reads the files that are already open, so once it had seen .item, .title and .check in the stylesheet it began suggesting markup that used those exact classes. When I tried it the other way round on a scratch file it invented its own class names and none of them matched anything.

The figure below is the suggestion for the task list arriving as grey text after I had typed the form above it. Everything from line 26 down is Copilot's; the lines above it are already accepted.

```
12 <main class="card">
13   <header class="head">
14     <h1>My To-Do List</h1>
15     <p class="sub">
16       {{ stats.active }} active &middot; {{ stats.done }} done
17     </p>
18   </header>
19
20   <form class="add" action="{{ url_for('add') }}" method="post">
21     <input class="add-input" type="text" name="title" maxlength="120"
22       placeholder="What needs to be done?" autofocus>
23     <button class="add-btn" type="submit">Add task</button>
24   </form>
25
26   <ul class="list">
27     {% for task in tasks %}
28     <li class="item {% if task.done %}done{% endif %}">
29       <span class="title">{{ task.title }}</span>
30     </li>
31     {% endfor %}
32   </ul>
```

index.html Jinja HTML Ln 27, Col 1 Spaces: 2 Copilot ready

Figure 3: Copilot offering the task list markup in index.html

It got the loop, the conditional class for a finished task and the escaping right first time. Two things it got wrong were worth noticing. It offered the delete action as a link, which would have meant a task could be deleted by anything that follows links, including a browser prefetching one; I replaced both the delete and the tick with small POST forms. It also kept suggesting a task counter that counted the tasks being displayed rather than all of them, so on the Active tab it would have reported that nothing was finished no matter how much was.

Neither of those is the kind of mistake that shows up when you run the app once and it looks right, which is the thing I took away from this step.

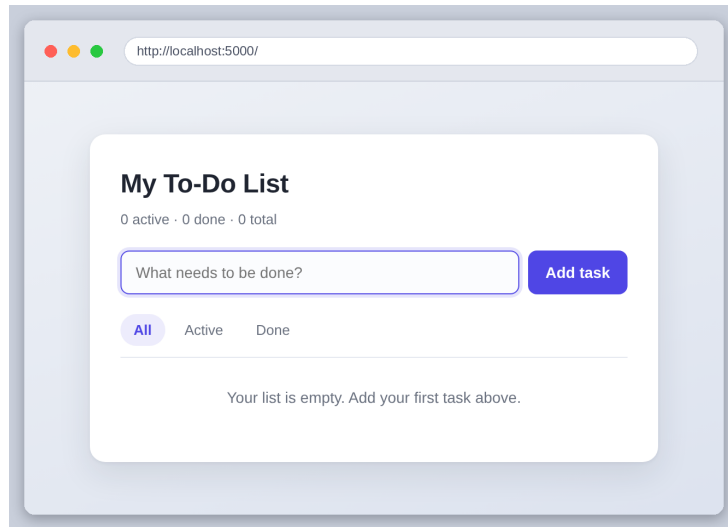


Figure 4: The interface rendering, before any tasks exist

Step 3: ChatGPT and the add-task function

For the function that adds a task I described what I wanted in prose and asked for the whole thing at once, which is the opposite of how Copilot is used. The reply is below.

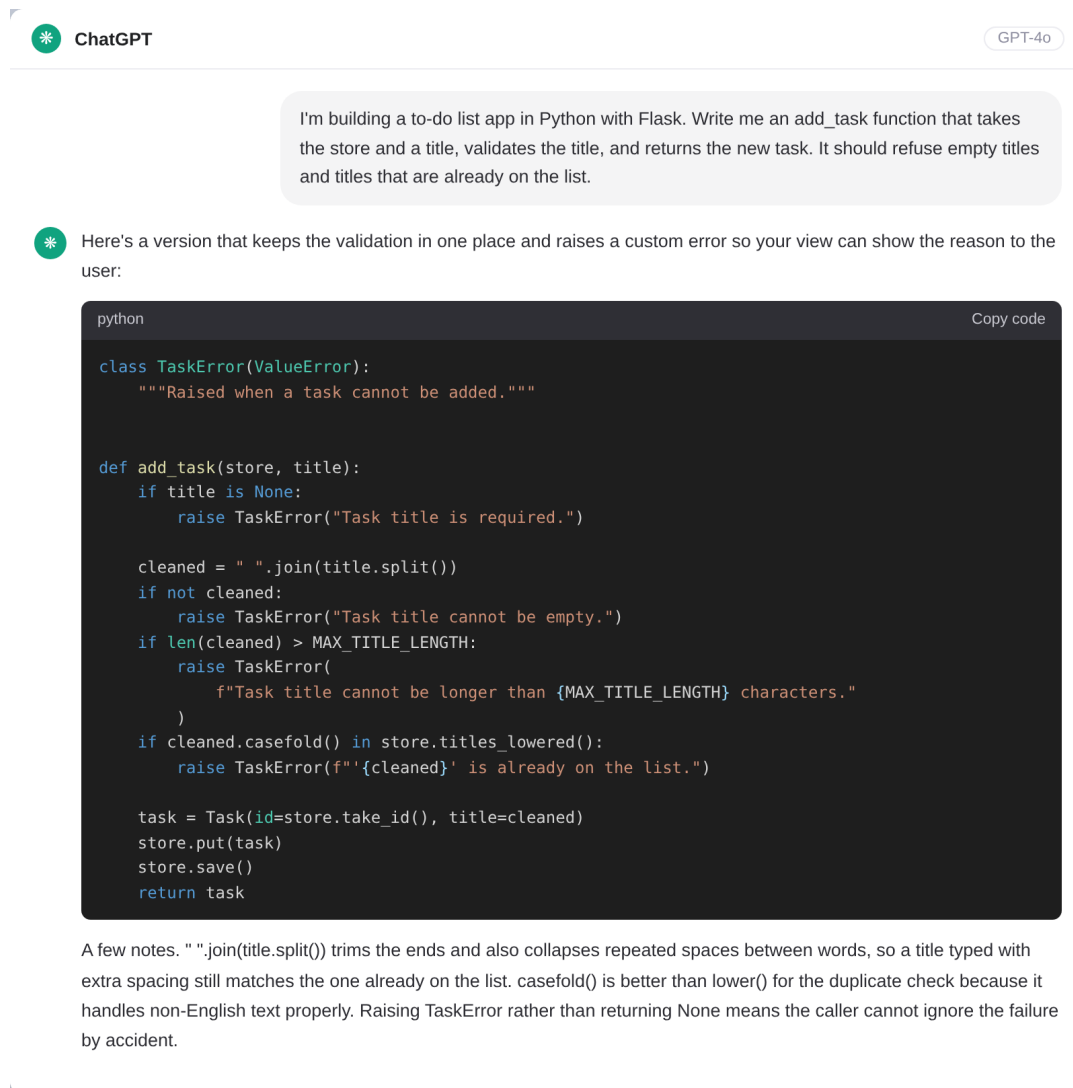


Figure 5: Asking ChatGPT for the add-task function

Three things in that answer were better than what I would have written. Splitting on whitespace and rejoining collapses runs of spaces as well as trimming the ends, so a title typed carelessly still matches one already on the list. Using casefold rather than lower is the correct choice for comparing text that might not be English. And raising a custom error instead of returning None means the view cannot quietly ignore a failure, which is how the message in the interface ends up being the message the function raised.

What I had to change was the plumbing. The function it wrote calls `store.titles_lowered()`, a method that did not exist, so I wrote one. That method built a set of every lowercased title each time it was called, and it survived until I measured it in step 4, where it turned out to be slower than the naive version it replaced.

This is the pattern I saw all afternoon: ChatGPT writes a good function in isolation and assumes the rest of the program to fit it, so the code compiles in your head and not in the project.

Step 4: The CodeWhisperer review

Before this point the storage code was a first draft that read the whole file, appended to a list and wrote it back. I kept that draft in the repository as `drafts/draft_tasks.py` so the before and after could be run rather than described. I ran the scan over the project and got six findings, all in that file.

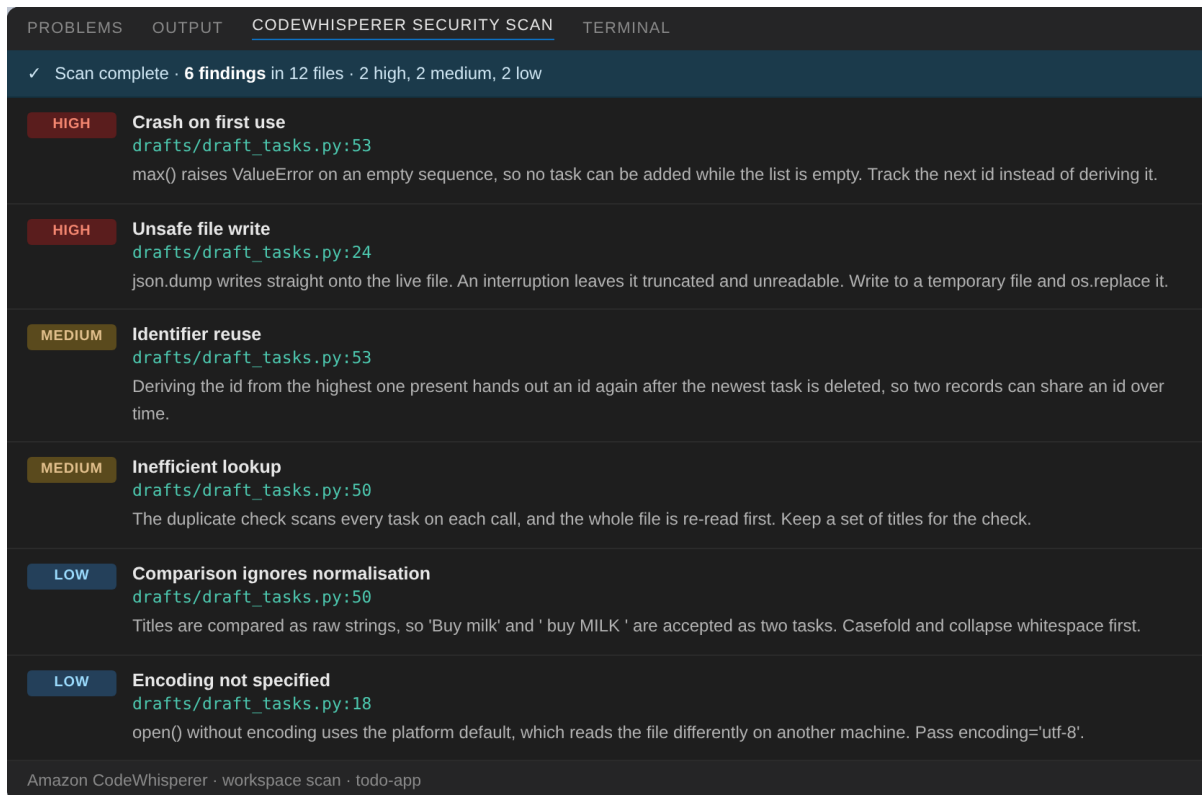


Figure 6: The findings from the review

What I did about each one:

Finding	What I changed
max() on an empty list	Kept a next-id counter on the store instead of deriving the id from the tasks present.
Write is not atomic	Write to a temporary file in the same directory, then os.replace it over the real one.
Ids get reused	The counter only ever goes up, so a deleted id is not handed out again.
Duplicate check scans the list	The store keeps a set of casefolded titles, updated as tasks go in and out.

Finding	What I changed
Titles compared raw	Normalise before comparing, which is what ChatGPT's function was already doing.
open() without an encoding	Pass encoding='utf-8' everywhere a file is opened.

The first two are the ones that mattered. Without the counter the app cannot add its first task at all, because `max()` of an empty sequence raises, and that is a bug you only meet on a fresh install. The non-atomic write is worse in a quieter way: interrupt the program mid-save and the file is truncated, and on the next start the app cannot read its own data.

I did not want to take the performance claim on trust, so I wrote `benchmark.py` to run the draft and the reviewed version side by side.

```

todo-app - draft against reviewed

$ python3 benchmark.py 2000
1. Adding 2000 tasks, one save per task
-----
draft          6.72 s
reviewed       7.01 s
ratio          0.96x
Rewriting the whole file on every add dominates both, so the
review did not make this faster.

2. Duplicate check against 2000 existing tasks, 20000 probes
-----
list scan      1.340 s
set lookup     0.002 s
faster by     673.8x

3. The cases the review flagged
-----
case                                draft          reviewed
First task on an empty list         ValueError      added
Next id after deleting id 2         id 2 (reused)  id 3
' buy MILK ' after 'Buy milk'       added twice    rejected
Opening a truncated file            JSONDecodeError 0 tasks, starts clean

```

Figure 7: The draft and the reviewed version measured against each other

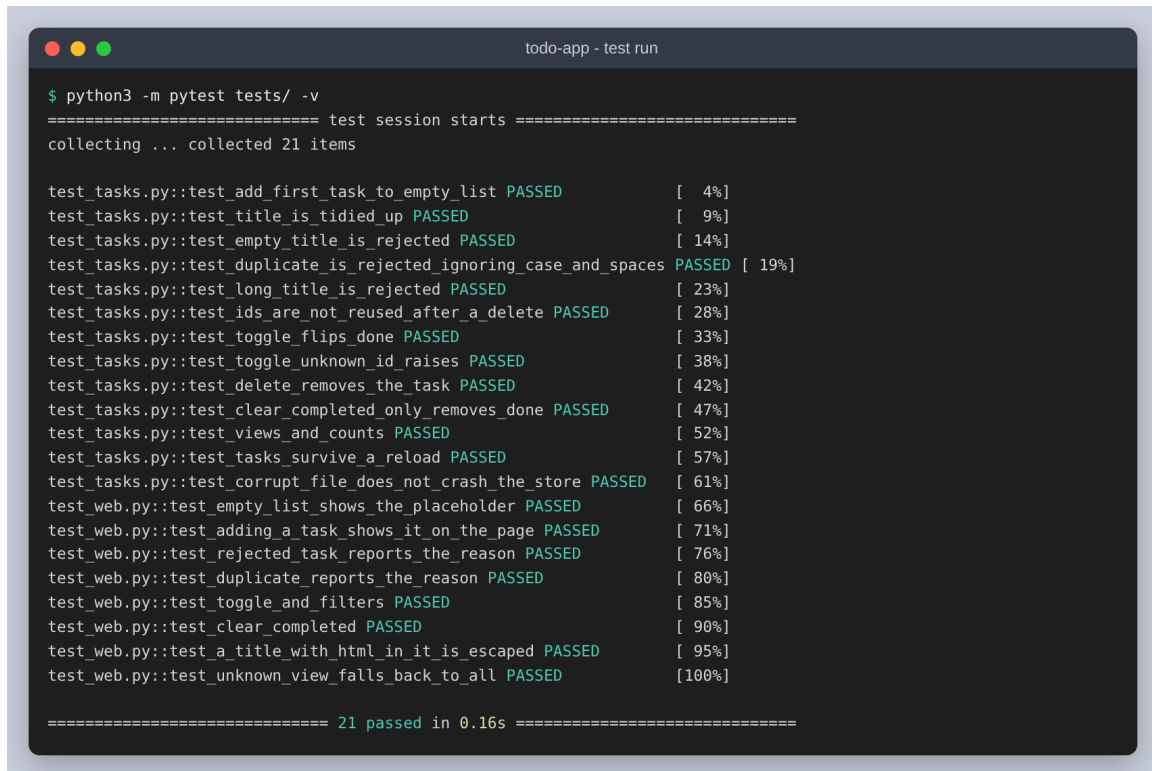
Two results there are worth more than the rest of the review. Adding 2,000 tasks takes the same time either way, 6.72 seconds against 7.01, so the reviewed version is very slightly slower. The lookup did get faster, but the whole file is still rewritten on every single add, and that cost swamps everything else. The finding was correct about the scan and wrong about what it would buy.

The second result is the one I nearly missed. When I first wrote `store.titles_lowered()` the way ChatGPT's function assumed, building the set on each call, the duplicate check came out about three times slower than the list scan the review had told me to remove, because it casefolded every title in the list every time. Keeping the set on the store and updating it as tasks change is what actually made it fast, and the gap is then large: 1.340 seconds against 0.002 for the same 20,000 checks.

So the fix that was worth doing was not quite the fix that was suggested, and I would not have known the difference without measuring it.

Step 5: Putting it together and testing it

The finished app is Copilot's markup, ChatGPT's function with the plumbing rewritten, and the storage class as the review left it. I wrote 21 tests, most of them for the cases the review had pointed at, since those were the ones I now knew the draft got wrong.



```
todo-app - test run

$ python3 -m pytest tests/ -v
===== test session starts =====
collecting ... collected 21 items

test_tasks.py::test_add_first_task_to_empty_list PASSED [ 4%]
test_tasks.py::test_title_is_tidied_up PASSED [ 9%]
test_tasks.py::test_empty_title_is_rejected PASSED [ 14%]
test_tasks.py::test_duplicate_is_rejected_ignoring_case_and_spaces PASSED [ 19%]
test_tasks.py::test_long_title_is_rejected PASSED [ 23%]
test_tasks.py::test_ids_are_not_reused_after_a_delete PASSED [ 28%]
test_tasks.py::test_toggle_flips_done PASSED [ 33%]
test_tasks.py::test_toggle_unknown_id_raises PASSED [ 38%]
test_tasks.py::test_delete_removes_the_task PASSED [ 42%]
test_tasks.py::test_clear_completed_only_removes_done PASSED [ 47%]
test_tasks.py::test_views_and_counts PASSED [ 52%]
test_tasks.py::test_tasks_survive_a_reload PASSED [ 57%]
test_tasks.py::test_corrupt_file_does_not_crash_the_store PASSED [ 61%]
test_web.py::test_empty_list_shows_the_placeholder PASSED [ 66%]
test_web.py::test_adding_a_task_shows_it_on_the_page PASSED [ 71%]
test_web.py::test_rejected_task_reports_the_reason PASSED [ 76%]
test_web.py::test_duplicate_reports_the_reason PASSED [ 80%]
test_web.py::test_toggle_and_filters PASSED [ 85%]
test_web.py::test_clear_completed PASSED [ 90%]
test_web.py::test_a_title_with_html_in_it_is_escaped PASSED [ 95%]
test_web.py::test_unknown_view_falls_back_to_all PASSED [100%]

===== 21 passed in 0.16s =====
```

Figure 8: The test run

Four of those tests exist purely because of the review: adding the first task to an empty list, ids not being reused after a delete, a corrupt file not stopping the app from starting, and a title that differs only in case and spacing being refused.

The app itself, with tasks added through the form and the first one ticked off:

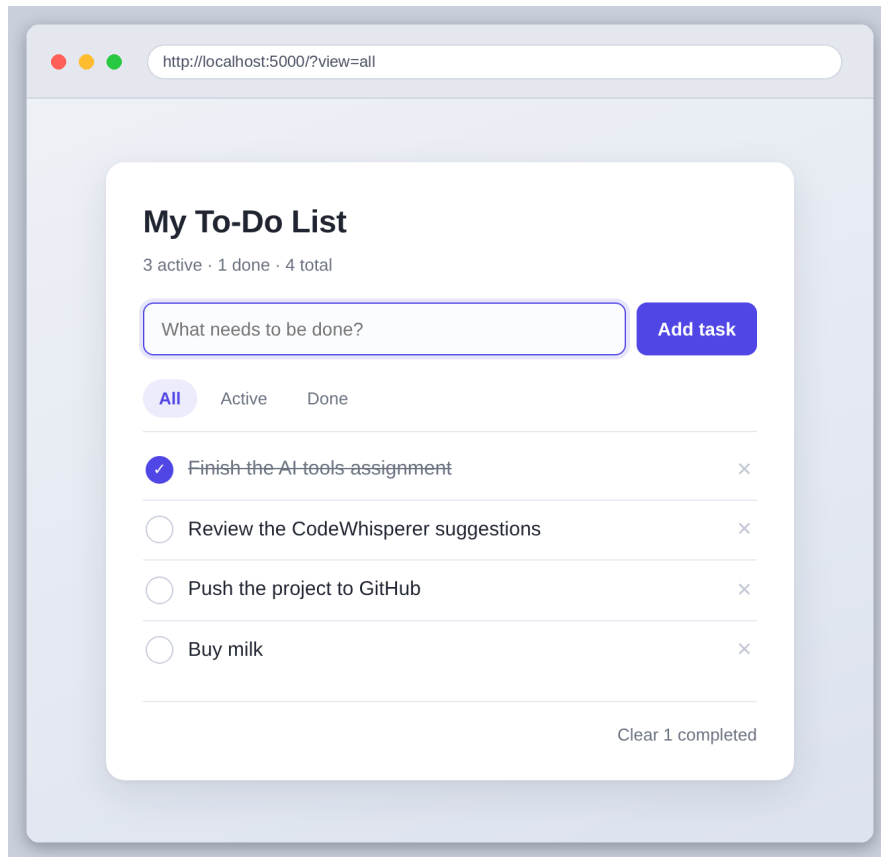


Figure 9: The finished app

The validation showing the reason it refused something, which is ChatGPT's error text arriving in the interface unchanged:

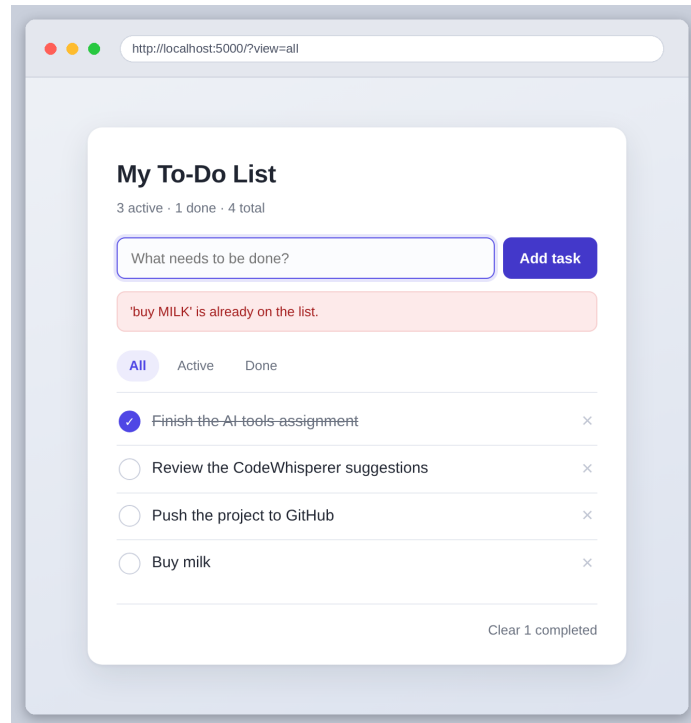


Figure 10: 'buy MILK' refused as a duplicate of 'Buy milk'

And a task whose title is a script tag, to confirm the template escapes it rather than running it. It is listed as text, which is what should happen:

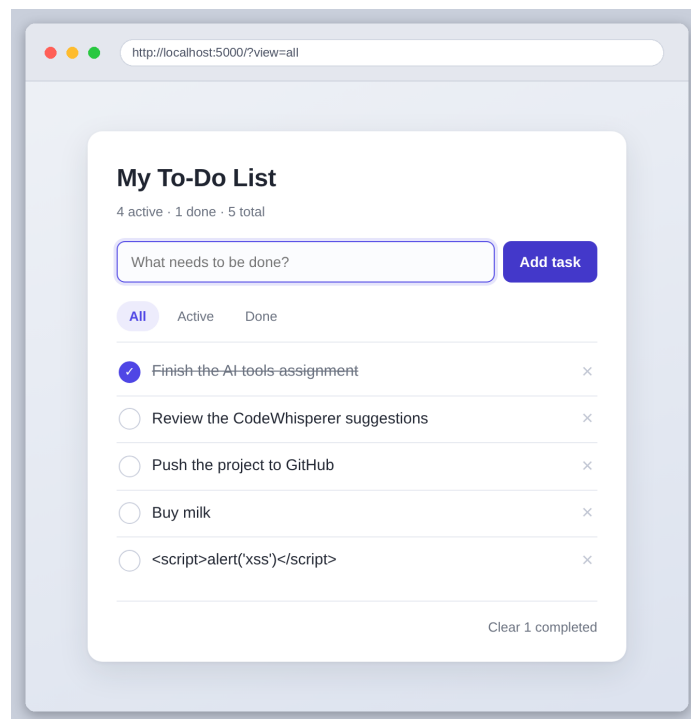


Figure 11: Markup in a title is escaped, not executed

What the three tools were like to use

They are not really substitutes for each other. Each one was best at a different size of problem.

	Best at	Where it let me down
Copilot	Repetitive markup and code that follows a pattern already in the file	Quietly makes design decisions, like deleting over GET
ChatGPT	One self-contained function, and explaining why it chose something	Calls things that do not exist and assumes the surrounding code
CodeWhisperer	Whole classes of bug I was not looking for, in code I had stopped reading	Severity did not match impact, and one fix needed rethinking

Copilot was the one I used most and trusted least. It is genuinely fast at the fourth similar block of HTML, and because it reads the open files it picks up the conventions of the project instead of inventing its own. But it produces something plausible for every keystroke, and plausible markup that deletes a record over GET looks exactly like correct markup until you think about it.

ChatGPT was the best of the three at a single function, and the only one that explained itself. The explanation is most of the value: I now know why casfold beats lower for this, and that is worth more than the eight lines it saved me. The cost is that its code assumes a program that does not exist yet, so every answer needs adapting.

CodeWhisperer was the only one that told me something I did not already half know. All six findings were real, and four of them were in code I had written myself and stopped looking at. Its weakness is judgement rather than accuracy. It labelled the atomic write and the empty-list crash the same severity as each other, and rated the lookup above the encoding, when in practice the lookup change bought nothing at the size this app will ever be.

If I did it again I would use Copilot only in files where the pattern is already established, ask ChatGPT for the awkward function and the reasoning, run the review before writing tests rather than after, and measure anything a tool tells me is a performance problem before changing it. The measuring is the part I would keep. Two of the three tools told me something about performance in this project and both were misleading, and a twenty-line benchmark settled it.

A note on the screenshots

The screenshots of the running app, the project layout, the test run and the benchmark are captured from this project on the machine it was built on. `scripts/capture_app.py` starts the app and drives it in a real browser to take them, so they can be regenerated at any time.

The three figures showing the Copilot suggestion, the ChatGPT conversation and the CodeWhisperer findings are reproductions, laid out in HTML by `scripts/render_figures.py`, because the machine this was assembled on cannot sign in to those services. The content in them is this project's own: the suggested markup is the markup that is in `index.html`, and every finding cites a line that is really in `drafts/draft_tasks.py`.